Construir un Chatbot para Tus Datos



Tiempo estimado necesario: 60 min

Introducción

En este proyecto, crearás un chatbot para tu propio archivo pdf utilizando Flask, un marco web popular, y LangChain, otro marco popular para trabajar con modelos de lenguaje grandes (LLMs). El chatbot que desarrolles no solo interactuará con los usuarios a través de texto, sino que también comprenderá y responderá preguntas relacionadas con el contenido de un documento específico.

¡Haz clic en el enlace de demostración a continuación para probar la aplicación final que crearás!

Prueba la aplicación de demostración

Al final de este proyecto, obtendrás una comprensión más profunda de los chatbots, el desarrollo de aplicaciones web utilizando Flask y Python, y el uso del marco LangChain para interpretar y responder a una amplia variedad de entradas de usuario. Y lo más importante, ¡habrás construido una aplicación de chatbot completa e impresionante!

The content of this lab is licensed under Apache 2.0



Una persona busca un documento en una enorme pila de papeles.

Antes de comenzar, veamos los elementos clave que utilizarás para construir tu aplicación.

Chatbots

Los chatbots son aplicaciones de software diseñadas para participar en conversaciones similares a las humanas. Pueden responder a entradas de texto de los usuarios y se utilizan ampliamente en varios dominios, incluidos el servicio al cliente, el comercio electrónico y la educación. En este proyecto, construirás un chatbot capaz de no solo involucrar a los usuarios en una conversación general, sino también de responder consultas basadas en un documento particular.

LangChain

LangChain es una herramienta versátil para construir aplicaciones de lenguaje impulsadas por IA. Proporciona diversas funcionalidades como recuperación de texto, resumido, traducción y muchas más, aprovechando modelos de lenguaje preentrenados. En este proyecto, integrarás LangChain en tu chatbot, dándole la

The content of this lab is licensed under Apache 2.0

capacidad de comprender y responder a diversas entradas de usuario de manera efectiva.

Flask

Flask es un marco web ligero y flexible para Python, conocido por su simplicidad y velocidad. Un marco web es un marco de software diseñado para apoyar el desarrollo de aplicaciones web, incluida la creación de servidores web y la gestión de solicitudes y respuestas HTTP.

Utilizarás Flask para crear el lado del servidor o backend de tu chatbot. Esto implica manejar los mensajes entrantes de los usuarios, procesar estos mensajes y enviar respuestas adecuadas de vuelta al usuario.

Rutas en Flask

Las rutas son una parte esencial del desarrollo web. Cuando tu aplicación recibe una solicitud de un cliente (típicamente un navegador web), necesita saber cómo manejar esa solicitud. Aquí es donde entra el enrutamiento.

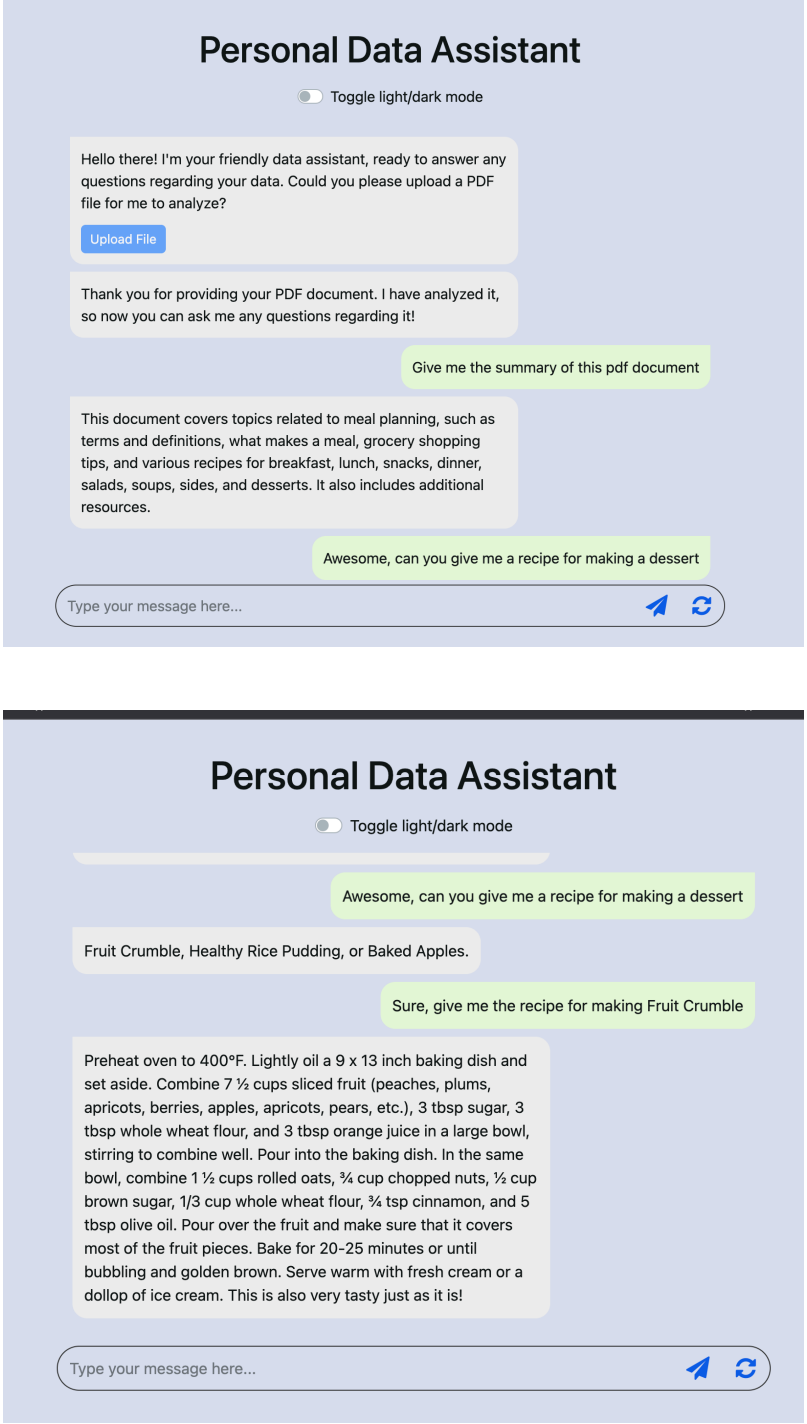
En Flask, las rutas se crean utilizando el decorador @app.route para vincular una función a una ruta URL. Cuando un usuario visita esa URL, se ejecuta la función asociada. En tu proyecto de chatbot, utilizarás rutas para manejar las solicitudes POST que llevan los mensajes de los usuarios y para procesar los datos del documento.

HTML - CSS - JavaScript

Se te proporciona un front-end de chatbot listo para usar, construido con HTML, CSS y JavaScript. HTML estructura el contenido web, CSS lo estiliza y JavaScript añade interactividad. Estas tecnologías crean una interfaz de chatbot visualmente atractiva e interactiva.

Aquí hay una instantánea de la interfaz:

The content of this lab is licensed under Apache 2.0



Interfaz del Chatbot

Objetivos de aprendizaje

Al final de este proyecto, serás capaz de:

- Explicar los conceptos básicos de Langchain y las aplicaciones de IA
- Configurar un entorno de desarrollo para construir un asistente utilizando Python Flask
- Implementar la funcionalidad de carga de PDF para permitir que el asistente comprenda los archivos de entrada de los usuarios

The content of this lab is licensed under Apache 2.0

- Integrar el asistente con modelos de código abierto para darle un alto nivel de inteligencia y la capacidad de comprender y responder a las solicitudes de los usuarios
- (Opcional) Desplegar el asistente de PDF en un servidor web para su uso por un público más amplio

Requisitos previos

Tener conocimientos básicos de HTML/CSS, JavaScript y Python es recomendable, pero no esencial. Cada paso del proceso y el código tendrán una explicación completa en este laboratorio.

Con esto en mente, ¡comencemos con tu proyecto!

The content of this lab is licensed under Apache 2.0

Configuración y comprensión de la interfaz de usuario

En este proyecto, el objetivo es crear un chatbot con una interfaz que permita la comunicación.

Primero, configuremos el entorno ejecutando el siguiente código:

bash

```
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

El frontend utilizará HTML, CSS y JavaScript. La interfaz de usuario será similar a muchos chatbots que ves y usas en línea. El código para la interfaz se proporciona y el enfoque de este proyecto guiado es conectar esta interfaz con el backend que maneja la carga de tus documentos personalizados e integrarlo con un modelo LLM para obtener respuestas personalizadas. El código proporcionado te ayudará a entender cómo interactúan el frontend y el backend, y a medida que lo revises, aprenderás sobre las partes importantes y cómo funciona, dándote una comprensión clara de cómo funciona el frontend y cómo crear esta simple página web.

Ejecuta los siguientes comandos para recuperar el proyecto, darle un nombre apropiado y finalmente moverte a ese directorio ejecutando lo siguiente:

bash

```
git clone https://github.com/ibm-developer-skills-network/wbphl-build_own_
mv wbphl-build_own_chatbot_without_open_ai build_chatbot_for_your_data
cd build_chatbot_for_your_data
```

Run

instalando los requisitos para el proyecto

The content of this lab is licensed under Apache 2.0

bash

```
pip install -r requirements.txt
```

Run

bash

```
pip install "langchain==0.3.27" "langchain-core==0.3.80" "langchain-commun
```

Run

Toma una taza de café, tomará de 5 a 10 minutos instalar los requisitos (Puedes continuar con este proyecto mientras se instalan los requisitos).

plaintext

```
     )  (
    (   ) )
     ) ( (
   _______)_
 .-'---------|
( C|/\/\/\/\/|
 '-./\/\/\/\/|
   '_________'
    '-------'
```

La siguiente sección ofrece una breve comprensión de cómo funciona el frontend.

HTML, CSS y JavaScript

El archivo `index.html` es responsable del diseño y la estructura de la interfaz web. Este archivo contiene el código para incorporar bibliotecas externas como JQuery, Bootstrap y FontAwesome Icons, así como el código CSS (`style.css`) y JavaScript (`script.js`) que controlan el estilo y la interactividad de la interfaz.

El archivo `style.css` es responsable de personalizar la apariencia visual de los componentes de la página. También gestiona la animación de carga utilizando keyframes de CSS. Los keyframes son una forma de definir los valores de una animación en varios puntos en el tiempo, permitiendo una transición suave entre diferentes estilos y creando animaciones dinámicas.

El archivo `script.js` es responsable de la interactividad y funcionalidad de la

The content of this lab is licensed under Apache 2.0

necesarias, como cambiar entre modo claro y oscuro, enviar mensajes y mostrar nuevos mensajes en la pantalla. Incluso permite a los usuarios grabar audio.

The content of this lab is licensed under Apache 2.0

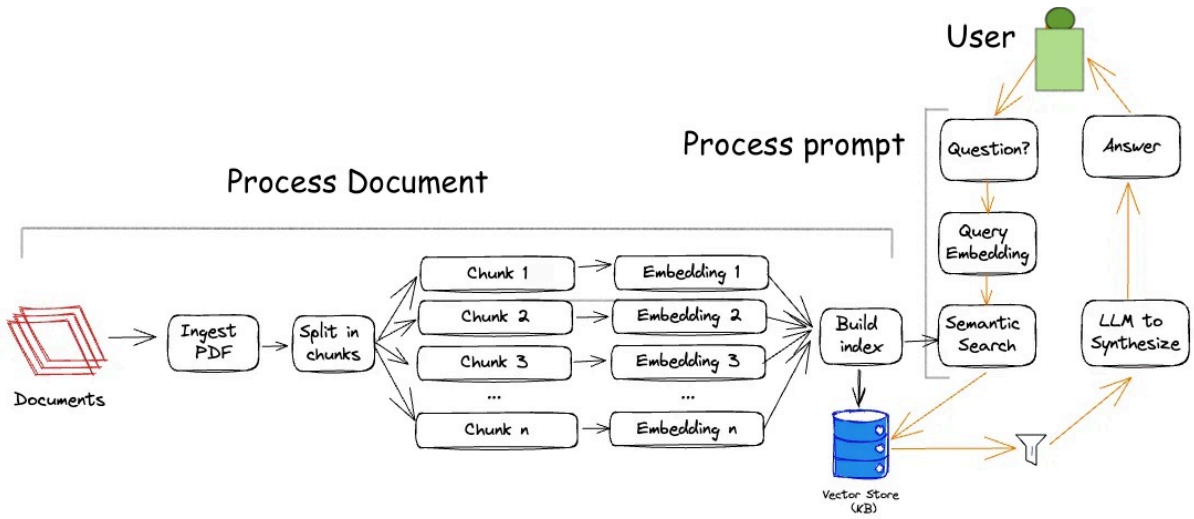
Entendiendo el trabajador: Procesamiento de documentos y gestión de conversaciones, parte 1

`worker.py` es parte de una aplicación de chatbot que procesa mensajes de usuarios y documentos. Utiliza la biblioteca `langchain`, que es una biblioteca de Python para construir aplicaciones de IA conversacional. Es responsable de configurar el modelo de lenguaje, procesar documentos PDF en un formato que se pueda utilizar para la recuperación de conversaciones y manejar las solicitudes de los usuarios para generar respuestas basadas en los documentos procesados. Aquí hay una descripción general del script:

Open worker.py in IDE

Tu tarea es completar los comentarios de `worker.py` con el código apropiado.

Desglosemos cada sección en el archivo worker. El worker.py está diseñado para proporcionar una interfaz conversacional que puede responder preguntas basadas en el contenido de un documento PDF dado.



El diagrama ilustra el procedimiento de procesamiento de documentos y recuperación de información, integrando sin problemas un modelo de lenguaje grande (LLM) para facilitar la tarea de responder preguntas. Todo el proceso ocurre en `worker.py`. *crédito de la imagen enlace*

The content of this lab is licensed under Apache 2.0

1. Inicialización `init_llm()`:
    - Configuración de variables de entorno: Se establece la variable de entorno para el token de la API de HuggingFace.
    - Carga del modelo de lenguaje: El modelo de lenguaje WatsonX se inicializa con parámetros especificados.
    - Carga de embeddings: Los embeddings se inicializan utilizando un modelo preentrenado.
2. Procesamiento de documentos `process_document(document_path)`: Esta función es responsable de procesar un documento PDF dado.
    - Carga del documento: El documento se carga utilizando PyPDFLoader.
    - División de texto: El documento se divide en fragmentos más pequeños usando RecursiveCharacterTextSplitter.
    - Creación de base de datos de embeddings: Se crea una base de datos de embeddings a partir de los fragmentos de texto usando Chroma.
    - Configuración de la cadena RetrievalQA: Se configura una cadena RetrievalQA para facilitar el proceso de respuesta a preguntas. Esta cadena utiliza el modelo de lenguaje inicializado y la base de datos de embeddings para responder preguntas basadas en el documento procesado.
3. Procesamiento de solicitudes de usuario `process_prompt(prompt)`: Esta función procesa la solicitud o pregunta de un usuario.
    - Recepción de la solicitud del usuario: El sistema recibe una solicitud del usuario (pregunta).
    - Consulta al modelo: Se consulta al modelo utilizando la cadena de recuperación, y genera una respuesta basada en el documento procesado y el historial de chat anterior.
    - Actualización del historial de chat: El historial de chat se actualiza con la nueva solicitud y respuesta.

The content of this lab is licensed under Apache 2.0

Profundizando en cada sección

> *IBM watsonX utiliza varios modelos de lenguaje, incluidos los modelos Llama de Meta, que han sido algunos de los modelos de lenguaje de código abierto más potentes publicados hasta ahora (en febrero de 2024).*

1. Inicialización `init_llm()`:

Este código es para configurar y usar un modelo de lenguaje de IA, de IBM watsonX:

1. **Configuración de credenciales**: Inicializa un diccionario con la URL del servicio y un token de autenticación (`"skills-network"`).

2. **Configuración de parámetros**: Establece parámetros del modelo como la generación máxima de tokens (`256`) y la temperatura (`0.1`, controlando la aleatoriedad).

3. **Inicialización del modelo**: Crea una instancia del modelo con un `model_id` específico, utilizando las credenciales y parámetros definidos anteriormente, y especifica `"skills-network"` como el ID del proyecto.

4. **Uso del modelo**: Inicializa una interfaz (`WatsonxLLM`) con el modelo configurado para la interacción.

Este script está específicamente configurado para un proyecto o entorno asociado con la "skills-network".

Completa el siguiente código en `worker.py` insertando los embeddings.

En este proyecto, no necesitas especificar tu propio `Watsonx_API` y `Project_id`. Solo puedes especificar `project_id="skills-network"` y dejar `Watsonx_API` en blanco.

Pero es importante notar que este método de acceso es exclusivo para este entorno de Cloud IDE. Si estás interesado en usar el modelo/API fuera de este entorno (por ejemplo, en un entorno local), hay instrucciones detalladas y más información disponibles en este tutorial.

The content of this lab is licensed under Apache 2.0

python

```python
# placeholder for Watsonx_API and Project_id incase you need to use the co
Watsonx_API = "Your WatsonX API"
Project_id= "Your Project ID"

# Function to initialize the language model and its embeddings
def init_llm():
    global llm_hub, embeddings

    logger.info("Initializing WatsonxLLM and embeddings...")

    # Llama Model Configuration
    MODEL_ID ="meta-llama/llama-4-maverick-17b-128e-instruct-fp8"
    WATSONX_URL = "https://us-south.ml.cloud.ibm.com"
    PROJECT_ID = "skills-network"

    # Use the same parameters as before:
    #   MAX_NEW_TOKENS: 256, TEMPERATURE: 0.1
    model_parameters = {
        # "decoding_method": "greedy",
        "max_new_tokens": 256,
        "temperature": 0.1,
    }

    # Initialize Llama LLM using the updated WatsonxLLM API
    llm_hub = WatsonxLLM(
        model_id=MODEL_ID,
        url=WATSONX_URL,
        project_id=PROJECT_ID,
        params=model_parameters
    )
    logger.debug("WatsonxLLM initialized: %s", llm_hub)

    #Initialize embeddings using a pre-trained model to represent the text
    embeddings =  # create object of Hugging Face Embeddings with (model_n

    logger.debug("Embeddings initialized with model device: %s", DEVICE)
```

▶ Haz clic aquí para ver la solución

The content of this lab is licensed under Apache 2.0

Comprendiendo el trabajador, parte 2

2. **Procesamiento de documentos:** la función `process_document` es responsable de procesar los documentos PDF. Utiliza el PyPDFLoader para cargar el documento, divide el documento en fragmentos utilizando el `RecursiveCharacterTextSplitter`, y luego crea un almacén de vectores (Chroma) a partir de los fragmentos del documento utilizando las incrustaciones del modelo de lenguaje. Este almacén de vectores se utiliza para crear una interfaz de recuperación, que se usa para crear un `ConversationalRetrievalChain`.

    - **Carga de documentos:** El documento PDF se carga utilizando la clase `PyPDFLoader`, que toma la ruta del documento como argumento. (Ejercicio: asignar PyPDFLoader(...) a `loader`)

    - **División de documentos:** El documento cargado se divide en fragmentos utilizando la clase `RecursiveCharacterTextSplitter`. Se pueden especificar el `chunk_size` y el `overlap`. (Ejercicio: asignar RecursiveCharacterTextSplitter(...) a `text_splitter`)

    - **Creación de almacén de vectores:** Se crea un almacén de vectores, que es una especie de índice, a partir de los fragmentos del documento utilizando las incrustaciones del modelo de lenguaje. Esto se realiza utilizando la clase `Chroma`.

    - **Configuración del sistema de recuperación:** Se configura un sistema de recuperación utilizando el almacén de vectores. Este sistema llama a un `ConversationalRetrievalChain`, utilizado para responder preguntas basadas en el contenido del documento.

Por hacer: completar las partes en blanco

The content of this lab is licensed under Apache 2.0

python

```python
# Function to process a PDF document
def process_document(document_path):
    global conversation_retrieval_chain

    logger.info("Loading document from path: %s", document_path)
    # Load the document
    loader =  # ---> use PyPDFLoader and document_path from the function i
    documents = loader.load()
    logger.debug("Loaded %d document(s)", len(documents))

    # Split the document into chunks, set chunk_size=1024, and chunk_overl
    text_splitter = # ---> use Recursive Character TextSplitter and specif
    texts = text_splitter.split_documents(documents)
    logger.debug("Document split into %d text chunks", len(texts))

    # Create an embeddings database using Chroma from the split text chunk
    logger.info("Initializing Chroma vector store from documents...")
    db = Chroma.from_documents(texts, embedding=embeddings)
    logger.debug("Chroma vector store initialized.")

    # Optional: Log available collections if accessible (this may be inter
    try:
        collections = db._client.list_collections()  # _client is internal
        logger.debug("Available collections in Chroma: %s", collections)
    except Exception as e:
        logger.warning("Could not retrieve collections from Chroma: %s", e

    # Build the QA chain, which utilizes the LLM and retriever for answeri
    conversation_retrieval_chain = RetrievalQA.from_chain_type(
        llm=llm_hub,
        chain_type="stuff",
        retriever=db.as_retriever(search_type="mmr", search_kwargs={'k': 6
        return_source_documents=False,
        input_key="question"
        # chain_type_kwargs={"prompt": prompt}  # if you are using a promp
    )
    logger.info("RetrievalQA chain created successfully.")
```

▶ Haga clic aquí para ver la solución

3. **Procesamiento de prompts (`process_prompt` function)**: Esta función maneja el prompt o pregunta de un usuario, recupera una respuesta basada en el contenido del documento PDF previamente procesado y mantiene un historial de chat. Hace lo siguiente:

The content of this lab is licensed under Apache 2.0

- Pasa el prompt y el historial de chat al objeto `ConversationalRetrievalChain`. `conversation_retrieval_chain` es la herramienta principal utilizada para consultar el modelo de lenguaje y obtener una respuesta basada en el contenido del documento PDF procesado.
- Agrega el prompt y la respuesta del bot al historial de chat.
- Devuelve la respuesta del bot.

Aquí hay un esqueleto de la función `process_prompt` para el ejercicio:

python

```python
# Function to process a user prompt
def process_prompt(prompt):
    global conversation_retrieval_chain
    global chat_history

    logger.info("Processing prompt: %s", prompt)
    # Query the model using the new .invoke() method
    output = conversation_retrieval_chain.invoke({"question": prompt, "cha
    answer = output["result"]
    logger.debug("Model response: %s", answer)

    # Update the chat history
    # TODO: Append the prompt and the bot's response to the chat history u
    # --> write your code here <--

    logger.debug("Chat history updated. Total exchanges: %d", len(chat_his

    # Return the model's response
    return answer
```

▶ Haz clic aquí para ver la solución

You also need to insert your LLM API key. Here's a demonstration to show you how to get your API key.

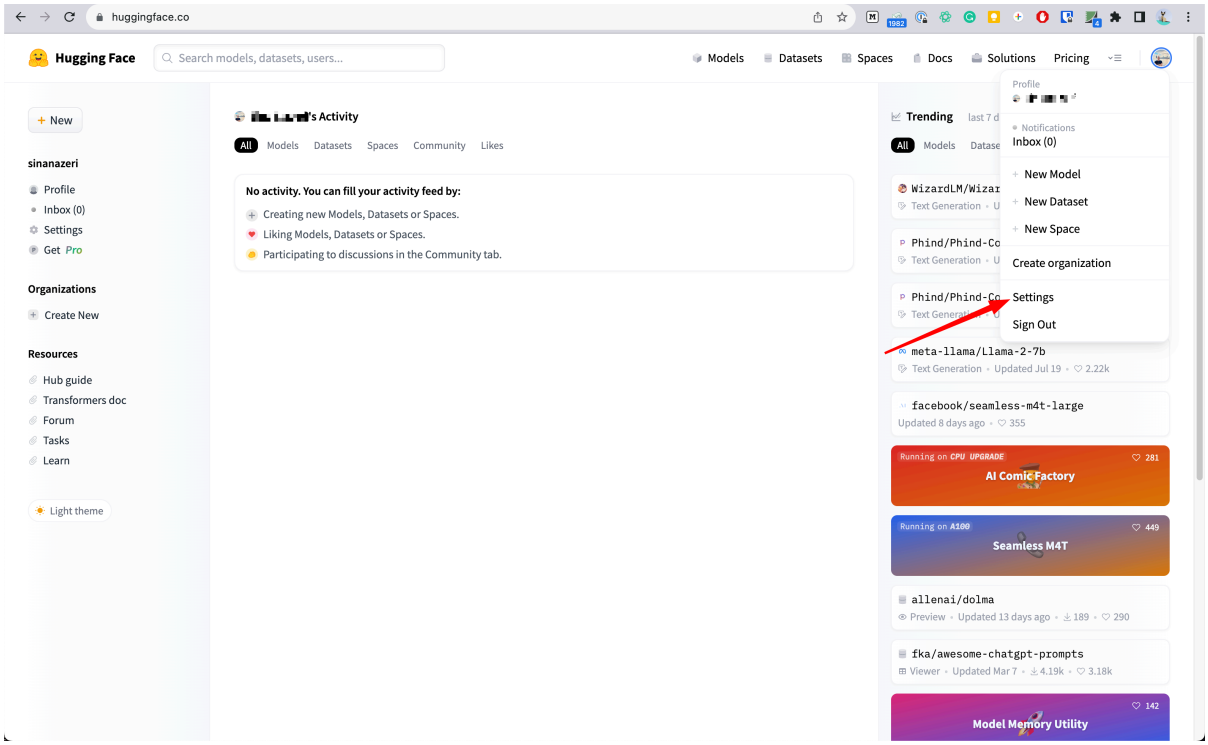
Initialize HuggingFace API key from your account with the following steps:

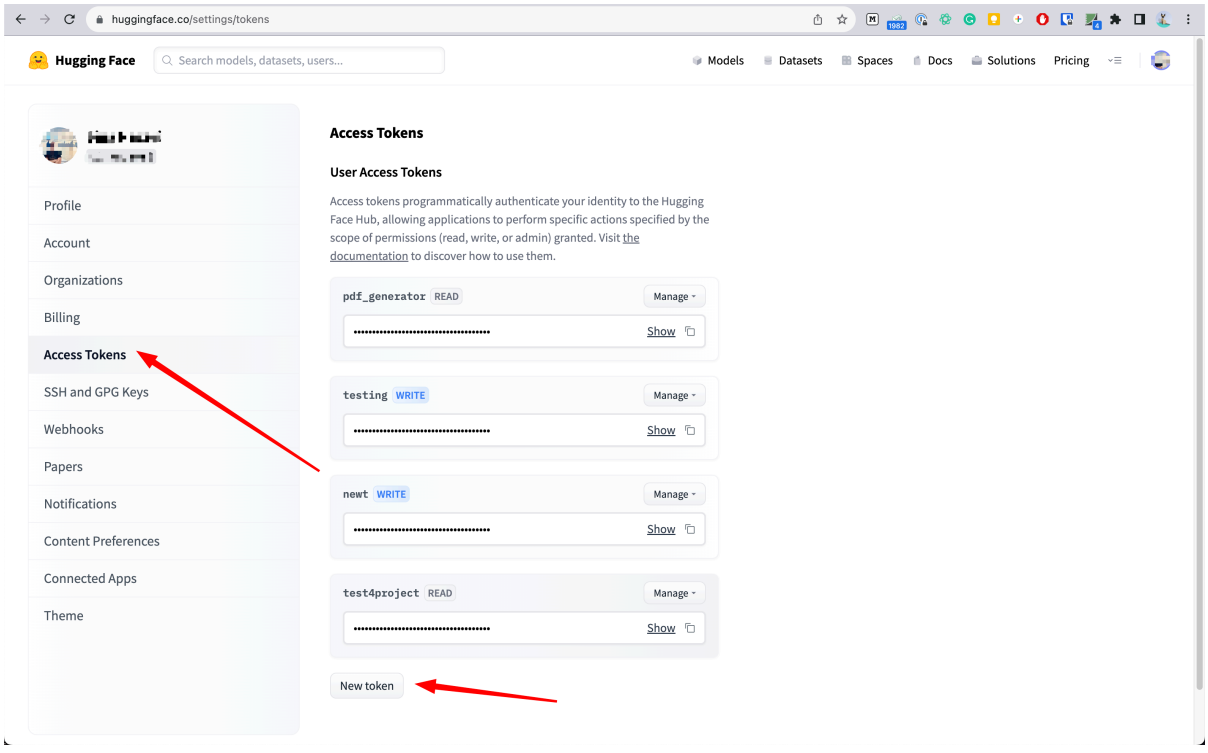
1. Go to the https://huggingface.co/
2. Log in to your account (or sign up free if it is your first time)

The content of this lab is licensed under Apache 2.0

3. Go to Settings -> Access Tokens -> click on New Token (refer image below)

4. Select either read or write option and copy the token





HuggingFace Token

▶ Click here to see the complete worker.py for huggingface version

The content of this lab is licensed under Apache 2.0

Construir un Chatbot para Tus Datos



Tiempo estimado necesario: 60 min

Introducción

En este proyecto, crearás un chatbot para tu propio archivo pdf utilizando Flask, un marco web popular, y LangChain, otro marco popular para trabajar con modelos de lenguaje grandes (LLMs). El chatbot que desarrolles no solo interactuará con los usuarios a través de texto, sino que también comprenderá y responderá preguntas relacionadas con el contenido de un documento específico.

¡Haz clic en el enlace de demostración a continuación para probar la aplicación final que crearás!

Prueba la aplicación de demostración

Al final de este proyecto, obtendrás una comprensión más profunda de los chatbots, el desarrollo de aplicaciones web utilizando Flask y Python, y el uso del marco LangChain para interpretar y responder a una amplia variedad de entradas de usuario. Y lo más importante, ¡habrás construido una aplicación de chatbot completa e impresionante!

The content of this lab is licensed under Apache 2.0



Una persona busca un documento en una enorme pila de papeles.

Antes de comenzar, veamos los elementos clave que utilizarás para construir tu aplicación.

Chatbots

Los chatbots son aplicaciones de software diseñadas para participar en conversaciones similares a las humanas. Pueden responder a entradas de texto de los usuarios y se utilizan ampliamente en varios dominios, incluidos el servicio al cliente, el comercio electrónico y la educación. En este proyecto, construirás un chatbot capaz de no solo involucrar a los usuarios en una conversación general, sino también de responder consultas basadas en un documento particular.

LangChain

LangChain es una herramienta versátil para construir aplicaciones de lenguaje impulsadas por IA. Proporciona diversas funcionalidades como recuperación de texto, resumido, traducción y muchas más, aprovechando modelos de lenguaje preentrenados. En este proyecto, integrarás LangChain en tu chatbot, dándole la

The content of this lab is licensed under Apache 2.0

capacidad de comprender y responder a diversas entradas de usuario de manera efectiva.

Flask

Flask es un marco web ligero y flexible para Python, conocido por su simplicidad y velocidad. Un marco web es un marco de software diseñado para apoyar el desarrollo de aplicaciones web, incluida la creación de servidores web y la gestión de solicitudes y respuestas HTTP.

Utilizarás Flask para crear el lado del servidor o backend de tu chatbot. Esto implica manejar los mensajes entrantes de los usuarios, procesar estos mensajes y enviar respuestas adecuadas de vuelta al usuario.

Rutas en Flask

Las rutas son una parte esencial del desarrollo web. Cuando tu aplicación recibe una solicitud de un cliente (típicamente un navegador web), necesita saber cómo manejar esa solicitud. Aquí es donde entra el enrutamiento.

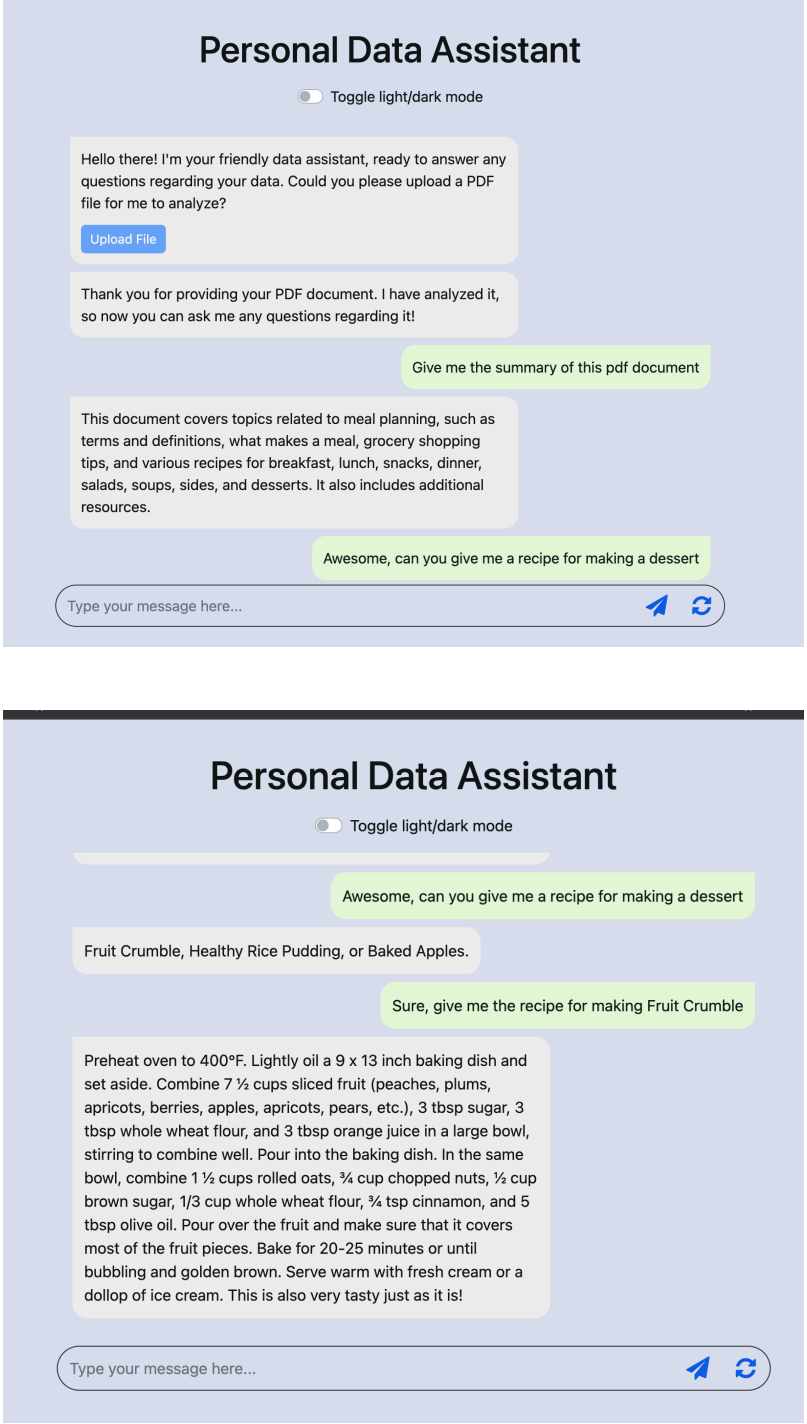
En Flask, las rutas se crean utilizando el decorador @app.route para vincular una función a una ruta URL. Cuando un usuario visita esa URL, se ejecuta la función asociada. En tu proyecto de chatbot, utilizarás rutas para manejar las solicitudes POST que llevan los mensajes de los usuarios y para procesar los datos del documento.

HTML - CSS - JavaScript

Se te proporciona un front-end de chatbot listo para usar, construido con HTML, CSS y JavaScript. HTML estructura el contenido web, CSS lo estiliza y JavaScript añade interactividad. Estas tecnologías crean una interfaz de chatbot visualmente atractiva e interactiva.

Aquí hay una instantánea de la interfaz:

The content of this lab is licensed under Apache 2.0



Interfaz del Chatbot

Objetivos de aprendizaje

Al final de este proyecto, serás capaz de:

- Explicar los conceptos básicos de Langchain y las aplicaciones de IA
- Configurar un entorno de desarrollo para construir un asistente utilizando Python Flask
- Implementar la funcionalidad de carga de PDF para permitir que el asistente comprenda los archivos de entrada de los usuarios

The content of this lab is licensed under Apache 2.0

- Integrar el asistente con modelos de código abierto para darle un alto nivel de inteligencia y la capacidad de comprender y responder a las solicitudes de los usuarios
- (Opcional) Desplegar el asistente de PDF en un servidor web para su uso por un público más amplio

Requisitos previos

Tener conocimientos básicos de HTML/CSS, JavaScript y Python es recomendable, pero no esencial. Cada paso del proceso y el código tendrán una explicación completa en este laboratorio.

Con esto en mente, ¡comencemos con tu proyecto!

Configuración y compren… →

The content of this lab is licensed under Apache 2.0